

2026-06-10-E-fork-mechanism

Workstream E — Fork-ALL-now mechanism + auto-merge-from-upstream

Status: Analysis / Design (PLANNING phase — READ-ONLY; nothing forked, swapped, committed, or pushed) **Date:** 2026-06-10 **Author role:** read-only analysis/design subagent **Evidence base:** root /Volumes/T7/Projects/HelixCode/.gitmodules (381 lines), live working-tree probes of cli_agents/*, constitution/scripts/ inventory. Every URL/path below is copied verbatim from .gitmodules (parsed via git config -f .gitmodules --get-regexp).

0. Scope & decisions (given)

- Fork EVERY cli_agents/* third-party submodule under the vasic-digital org with prefix caf- (cli-agent-fork) → git@github.com:vasic-digital/caf-<name>.git.
 - Swap each submodule’s url in .gitmodules to our fork, then git submodule sync.
 - Recursively: any **nested 3rd-party** submodule gets the same treatment; nested **own-org** deps must resolve to our root-level versions (CONST-051 forbids nested own-org chains — pull to root).
 - Regularly fetch + merge upstream main/master into each fork.
 - Scripts are reusable / decoupled / generic / parameterized; live in constitution/scripts/.
 - Constraints: §11.4.113 absolute no-force-push (merge-onto-latest-main only); §11.4.30 gitignore hygiene — only minimal files (.gitignore, governance siblings) may be committed to forks; we do **not** modify upstream application code.
-

1. Fork mapping table (the 3rd-party set)

.gitmodules declares **50** cli_agents/* submodule entries. Of these: - **49** are third-party GitHub upstreams → **fork** to vasic-digital/caf-<name>. - **1** (claude-code-source) is already an own/private GitLab mirror → **DO NOT fork** (flagged below). - **1 nested** third-party submodule discovered inside cli_agents/cline/ → also forked → **50 forks total**.

1.1 Top-level cli_agents/* → fork (49)

#	submodule path	upstream url (verbatim)	proposed fork
1	cli_agents/agent-deck	git@github.com:asheshgoplani/agent-deck.git	git@github.com:vasic-digital/caf-agent-deck.git
2	cli_agents/aiagent		

#	submodule path	upstream url (verbatim)	proposed fork
		git@github.com:Xiaoccer/ AIAgent.git	git@github.com:vasic- digital/caf-aiagent.git
3	cli_agents/aichat	git@github.com:sigoden/aichat.git	git@github.com:vasic- digital/caf-aichat.git
4	cli_agents/aichat-llm- functions	git@github.com:sigoden/llm- functions.git	git@github.com:vasic- digital/caf-aichat-llm- functions.git
5	cli_agents/aider	git@github.com:Aider-AI/aider.git	git@github.com:vasic- digital/caf-aider.git
6	cli_agents/amazon-q	git@github.com:aws/amazon-q- developer-cli.git	git@github.com:vasic- digital/caf-amazon- q.git
7	cli_agents/bridle	git@github.com:jeremylongshore/ claude-code-plugins-plus-skills.git	git@github.com:vasic- digital/caf-bridle.git
8	cli_agents/claude-code	git@github.com:anthropics/claude- code.git	git@github.com:vasic- digital/caf-claude- code.git
9	cli_agents/claude- squad	git@github.com:smtg-ai/claude- squad.git	git@github.com:vasic- digital/caf-claude- squad.git
10	cli_agents/cli-agent	git@github.com:NathanGr33n/ CLI_Tool.git	git@github.com:vasic- digital/caf-cli- agent.git
11	cli_agents/cline	git@github.com:cline/cline.git	git@github.com:vasic- digital/caf-cline.git
12	cli_agents/codai	git@github.com:meysamhadeli/ codai.git	git@github.com:vasic- digital/caf-codai.git
13	cli_agents/codename- goose	git@github.com:jgenerali/codename- goose.git	git@github.com:vasic- digital/caf-codename- goose.git
14	cli_agents/codex	git@github.com:openai/codex.git	git@github.com:vasic- digital/caf-codex.git
15	cli_agents/codex-skills	git@github.com:openai/skills.git	git@github.com:vasic- digital/caf-codex- skills.git
16	cli_agents/conduit	git@github.com:lostintangent/ conduit-release.git	git@github.com:vasic- digital/caf-conduit.git
17	cli_agents/copilot-cli	git@github.com:github/copilot-cli.git	git@github.com:vasic- digital/caf-copilot- cli.git
18	cli_agents/crush	git@github.com:charmbracelet/ crush.git	git@github.com:vasic- digital/caf-crush.git
19	cli_agents/deepseek-cli	git@github.com:holasoymalva/ deepseek-cli.git	

#	submodule path	upstream url (verbatim)	proposed fork
			git@github.com:vasic-digital/caf-deepseek-cli.git
20	cli_agents/deepseek-cli-youkpan	git@github.com:youkpan/deepseek-cli.git	git@github.com:vasic-digital/caf-deepseek-cli-youkpan.git
21	cli_agents/fauxpilot	git@github.com:fauxpilot/fauxpilot.git	git@github.com:vasic-digital/caf-fauxpilot.git
22	cli_agents/forgue	git@github.com:antinomyhq/forgue.git	git@github.com:vasic-digital/caf-forgue.git
23	cli_agents/gemini-cli	git@github.com:google-gemini/gemini-cli.git	git@github.com:vasic-digital/caf-gemini-cli.git
24	cli_agents/get-shit-done	git@github.com:glittercowboy/get-shit-done.git	git@github.com:vasic-digital/caf-get-shit-done.git
25	cli_agents/git-mcp	git@github.com:idosal/git-mcp.git	git@github.com:vasic-digital/caf-git-mcp.git
26	cli_agents/gpt-engineer	git@github.com:AntonOsika/gpt-engineer.git	git@github.com:vasic-digital/caf-gpt-engineer.git
27	cli_agents/gptme	git@github.com:ErikBjare/gptme.git	git@github.com:vasic-digital/caf-gptme.git
28	cli_agents/junie	git@github.com:JetBrains/junie.git	git@github.com:vasic-digital/caf-junie.git
29	cli_agents/mistral-code	git@github.com:Wylgrif/Mistral-code.git	git@github.com:vasic-digital/caf-mistral-code.git
30	cli_agents/multiagent-coding	git@github.com:Danau5tin/multi-agent-coding-system.git	git@github.com:vasic-digital/caf-multiagent-coding.git
31	cli_agents/nanocoder	git@github.com:Nano-Collective/nanocoder.git	git@github.com:vasic-digital/caf-nanocoder.git
32	cli_agents/noi	git@github.com:lencx/Noi.git	git@github.com:vasic-digital/caf-noi.git
33	cli_agents/octogen	git@github.com:dbpunk-labs/octogen.git	git@github.com:vasic-digital/caf-octogen.git
34	cli_agents/open-interpreter	git@github.com:KillianLucas/open-interpreter.git	git@github.com:vasic-digital/caf-open-interpreter.git
35	cli_agents/plandex	git@github.com:plandex-ai/plandex.git	git@github.com:vasic-digital/caf-plandex.git

#	submodule path	upstream url (verbatim)	proposed fork
36	cli_agents/postgres-mcp	git@github.com:timescale/pg-aiguide.git	git@github.com:vasic-digital/caf-postgres-mcp.git
37	cli_agents/qwen-code	git@github.com:QwenLM/qwen-code.git	git@github.com:vasic-digital/caf-qwen-code.git
38	cli_agents/shai	git@github.com:ovh/shai.git	git@github.com:vasic-digital/caf-shai.git
39	cli_agents/snow-cli	git@github.com:MayDay-wpf/snow-cli.git	git@github.com:vasic-digital/caf-snow-cli.git
40	cli_agents/spec-kit	git@github.com:github/spec-kit.git	git@github.com:vasic-digital/caf-spec-kit.git
41	cli_agents/superset	git@github.com:superset-sh/superset.git	git@github.com:vasic-digital/caf-superset.git
42	cli_agents/swe-agent	git@github.com:princeton-nlp/SWE-agent.git	git@github.com:vasic-digital/caf-swe-agent.git
43	cli_agents/taskweaver	git@github.com:microsoft/TaskWeaver.git	git@github.com:vasic-digital/caf-taskweaver.git
44	cli_agents/ui-ux-pro-max	git@github.com:nextlevelbuilder/ui-ux-pro-max-skill.git	git@github.com:vasic-digital/caf-ui-ux-pro-max.git
45	cli_agents/vtcode	git@github.com:vinhnx/vtcode.git	git@github.com:vasic-digital/caf-vtcode.git
46	cli_agents/warp	git@github.com:warpdotdev/Warp.git	git@github.com:vasic-digital/caf-warp.git
47	cli_agents/x-cmd	git@github.com:x-cmd/x-cmd.git	git@github.com:vasic-digital/caf-x-cmd.git
48	cli_agents/xela-cli	git@github.com:xelauvas/codeclau.git	git@github.com:vasic-digital/caf-xela-cli.git
49	cli_agents/zeroshot	git@github.com:covibes/zeroshot.git	git@github.com:vasic-digital/caf-zeroshot.git

1.2 Nested 3rd-party submodule → fork (1)

#	nested path	upstream url (verbatim)	proposed fork	note
50	cli_agents/ cline → evals/ cline-bench	https://github.com/cline/ cline-bench.git	git@github.com:vasic-digital/caf-cline-bench.git	HTTPS upstream (Rule 3 — must be SSH in our fork). After <code>caf-cline</code> exists, edit <code>caf-</code>

#	nested path	upstream url (verbatim)	proposed fork	note
				cline's .gitmodules to point evals/ cline-bench → caf- cline-bench (recursive swap, committed to the fork — a permitted minimal change).

Fork count total: 50 (49 top-level + 1 nested).

1.3 Flagged anomalies (do NOT fork / special-case)

path	url	flag	action
cli_agents/claude-code-source	git@gitlab.com:milos85vasic/ccode-private.git	Already an own/private GitLab mirror (operator milos85vasic), not a public 3rd-party upstream.	Skip fork. It's controlled. The scripts must skip any url whose path is gitlab. It's our own name, whose path already resolves to our own. Optionally re-run under vasic/digital-lab for a different scope for the pass.
cli_agents_resources/OpenAI-Cookbook	org-14957082@github.com:openai/openai-cookbook.git	Org-prefixed SSH alias (org-14957082@) — a per-machine ~/.ssh/config GitHub-org SSH identity, NOT a path. Lives in cli_agents_resources/, out of scope (resources, not agents).	Not forked in the workflow. Only so the path is not mistaken for a different fork scripts must normalize on NNNN@github → github before deriving owner/repo

Nesting depth finding: of the 6 representative agents inspected (claude-code, qwen-code, aider, plandex, crush, gemini-cli) **none** has a nested .gitmodules. A full sweep (find cli_agents -maxdepth 2 -name .gitmodules) found exactly one nested manifest — cli_agents/cline/.gitmodules (depth 1, the single cline-bench entry above). Max recursion depth observed across the fleet = **1**. No nested **own-org** chains exist under cli_agents/* (all own-org deps already live flat at submodules/<name>/ per the

existing layout), so CONST-051's pull-to-root rule has **no** `cli_agents` violations to remediate — but the scripts must still enforce it generically for future nesting.

2. Reusable script designs (constitution/scripts/)

All three follow the existing house style observed in `constitution/scripts/`
`codegraph_update.sh`: `#!/usr/bin/env bash, set -uo pipefail,`
`SCRIPT_DIR/CONST_ROOT` resolution, anti-bluff PASS that observes real post-state (not just CLI exit code), documented exit codes, append-only Status ledger. **Nothing is hardcoded** — org, prefix, source dir, providers, remotes are all flags/env with defaults.

Common parameter surface (shared `lib_caf_lib.sh`, sourced by all three):

<code>--org <name></code>	default: <code>vasic-digital</code>	(env <code>CAF_ORG</code>)
<code>--prefix <str></code>	default: <code>caf-</code>	(env <code>CAF_PREFIX</code>)
<code>--src-dir <path></code>	default: <code>cli_agents</code>	(env <code>CAF_SRC_DIR</code>)
<code>--gitmodules <path></code>	default: <code><repo-root>/ .gitmodules</code>	(env <code>CAF_GITMODU</code>)
<code>--providers <csv></code>	default: <code>github,gitlab</code>	(env <code>CAF_PROVIDE</code>)
<code>--gitlab-group <name></code>	default: <code>vasic-digital</code>	(env <code>CAF_GITLAB_</code>)
<code>--branch <name></code>	default: <code>"</code> → auto-detect remote HEAD (main master	
<code>--dry-run</code>	print planned actions, mutate nothing (default for	
<code>--only <name[,name]></code>	restrict to a subset (testing / retry)	
<code>--exclude <name[,name]></code>	skip-list (defaults include <code>claude-code-source</code>)	
<code>--recursive</code>	follow nested 3rd-party submodules (depth-limited,	

`caf_lib.sh` helpers: `caf_normalize_url()` (strip `org-NNNN@`, host-normalize, derive owner/repo), `caf_is_own_org()` (true if url already in `vasic-digital/HelixDevelopment/...` → skip-or-pull-to-root), `caf_fork_name()` (`<prefix><submodule-dir-name>`), `caf_log()` (append to `docs/caf/Status.md` + stdout), `caf_detect_default_branch()` (`git ls-remote --symref <url> HEAD`).

2.1 `caf_fork_all.sh` — fork-or-create + remote wiring

Purpose: For every submodule under `--src-dir`, create `<org>/<prefix><name>` on each provider, seed it from upstream, and wire remotes (`origin=our fork, upstream=original`). Idempotent: re-running detects an existing fork and skips creation.

Inputs: the common surface above. Reads `--gitmodules` for `path+url` pairs filtered to `--src-dir`.

Per-submodule sequence (GitHub primary):

```
url=<verbatim from .gitmodules>
norm=$(caf_normalize_url "$url")           # org-NNNN@ stripped, → owner/
caf_is_own_org "$norm" && { caf_log SKIP own-org "$name"; continue; }
in_exclude "$name" && { caf_log SKIP excluded "$name"; continue; }
fork=$(caf_fork_name "$name")              # caf-<name>
```

```
# (a) fork-or-create on GitHub
if gh repo view "$ORG/$fork" >/dev/null 2>&1; then
    caf_log EXISTS "$ORG/$fork"
```

```

else
    # gh fork keeps the upstream link; fall back to create+mirror-push for
    # non-forkable sources (HTTPS-only, archived, or self-hosted upstream)
    if ! gh repo fork "$norm" --org "$ORG" --fork-name "$fork" --clone=fal
        gh repo create "$ORG/$fork" --private --disable-wiki
        tmp=$(mktemp -d); git clone --mirror "$url" "$tmp/.git"
        git -C "$tmp/.git" push --mirror "git@github.com:$ORG/$fork.git"
        rm -rf "$tmp"
    fi
fi

# (b) GitLab mirror of the same fork (dual-remote)
if provider_enabled gitlab; then
    glab repo view "$GITLAB_GROUP/$fork" >/dev/null 2>&1 \
        || glab repo create "$GITLAB_GROUP/$fork" --private
fi

# (c) record mapping for the swap + merge steps
echo "$name|$url|git@github.com:$ORG/$fork.git|git@gitlab.com:$GITLAB_GROU

```

Error handling: every gh/glab call wrapped; non-zero → `caf_log FAIL "$name"` "\$rc" and continue (one bad repo never aborts the batch). Rate-limit aware: on gh HTTP 403/429 the loop sleeps with exponential backoff (`gh api rate_limit` polled), resumable via `--only` from the partial `MAP_FILE`. Mirror-push initial seed targets an **empty** fork (no existing refs to overwrite) so it does not violate §11.4.113 (which forbids overwriting *existing* remote history).

Anti-bluff Challenge (`caf_fork_all.challenge.sh`): bootstrap a throwaway temp dir with a tiny fixture `.gitmodules` pointing at one small public repo (e.g. `octocat/Hello-World`) under `--src-dir=fixtures`; run with a disposable test org/group (or `--dry-run` asserting the exact gh/glab command lines emitted); on a live run assert `gh repo view <org>/caf-hello-world` exits 0 AND `git ls-remote git@github.com:<org>/caf-hello-world.git` returns refs AND the upstream remote resolves to the original. Captures stdout under `docs/qa/<run-id>/`. Teardown deletes the test fork. A PASS observes the **real remote**, never just the CLI exit code (§107/§11.4.5).

2.2 `caf_update_merge.sh` — **fetch + merge-onto-latest upstream**

Purpose: Keep every fork current by merging upstream `main/master` into the fork, fast-forward-preferred, **never force** (§11.4.113). Runs on cadence (weekly floor per §11.4.45 / §11.4.80 pattern).

Inputs: common surface + `--workdir <path>` (scratch clones dir, default `$(mktemp -d)`), `--strategy ff-only|merge` (default merge, union-preserving), `--push/--no-push` (default `--push`).

Per-fork loop (over `MAP_FILE` from §2.1, or re-derived from `.gitmodules`):

```

git clone "$fork_origin" "$wd/$fork"          # our fork
cd "$wd/$fork"
git remote add upstream "$upstream_url" 2>/dev/null || git remote set-url
git fetch --all --prune --tags
base=$(caf_detect_default_branch upstream) # main|master
git checkout "$base"

```

```
# §11.4.113 merge-onto-latest-main: base = latest fork main; merge upstream
if git merge --ff-only "upstream/$base" 2>/dev/null; then
    caf_log FF "$fork" "$base"
else
    git merge --no-edit "upstream/$base" || { caf_log CONFLICT "$fork"; re
    caf_log MERGE "$fork" "$base"
fi
# fan-out push to ALL configured provider remotes (fast-forward only)
for remote in $(provider_remotes "$fork"); do
    git push "$remote" "$base" || caf_log PUSHFAIL "$fork" "$remote"
done
```

Error handling / conflict policy: merge conflicts are **never** auto-resolved with `-s ours/-X theirs` (§11.4.113 step 4, §11.4.6 no-guessing). A conflicted fork is logged to `docs/caf/conflicts/<fork>_<ts>.md` with the conflicting paths and parked for operator review; the loop continues with the next fork (zero-idle, §11.4.94). Because the fork's origin/`<base>` is always the merge base and the merge commit descends from it, every push is a **fast-forward** → no force ever needed. If an upstream rewrote its history (rare), the script logs `UPSTREAM-REWRITE` and parks — it does **not** force.

Anti-bluff Challenge (`caf_update_merge.challenge.sh`): create a throwaway “upstream” repo + a “fork” clone in temp; add a commit to upstream; run `caf_update_merge.sh --only fixture --workdir <tmp>`; assert the fork's main now contains the upstream commit SHA (`git log grep`), assert `git rev-parse fork/main` is a fast-forward descendant of the pre-merge tip, and assert **no** `--force` appears in the captured git trace (`GIT_TRACE log` scanned). Then plant a conflicting edit on both sides and assert the script logs `CONFLICT + parks` (does not silently resolve). Evidence under `docs/qa/<run-id>/`.

2.3 `caf_resolve_nested_submodules.sh` — recursive nested-dep handler

Purpose: Enforce the recursive rule: inside each fork (or each checked-out submodule), find nested submodules; **3rd-party** nested deps get forked (delegated to `caf_fork_all.sh` with `--src-dir` set to the nested manifest's dir) and the fork's nested `.gitmodules` rewritten to our fork URL; **own-org** nested deps are flagged as `CONST-051` violations and reported for pull-to-root (the script does **not** silently rewrite own-org chains — it emits the remediation plan).

Inputs: common surface + `--depth <n>` (default 3) + `--repo <path-or-fork>`.

Sequence:

```
scan_for_nested() {
    local dir="$1" depth="$2"
    [ "$depth" -le 0 ] && return
    [ -f "$dir/.gitmodules" ] || return
    git config -f "$dir/.gitmodules" --get-regexp '\.url$' | while read -r
        norm=$(caf_normalize_url "$url")
        if caf_is_own_org "$norm"; then
            caf_log CONST051 "$dir" "$url"          # nested own-org chain → m
            echo "$dir|$url|PULL_TO_ROOT" >> "$NESTED_REPORT"
        else
            fork=$(caf_fork_name "$(basename "$key") .url" | sed 's/^submodu
            echo "$dir|$url|FORK:$ORG/$fork" >> "$NESTED_REPORT"
```



```

        # delegate the actual fork to caf_fork_all (idempotent)
    fi
done
# recurse one level deeper
}

```

For cline: scan finds evals/cline-bench → HTTPS 3rd-party → emits FORK: vasic-digital/caf-cline-bench and (after caf-cline-bench exists) rewrites caf-cline's evals/cline-bench url to git@github.com:vasic-digital/caf-cline-bench.git (SSH per Rule 3), committed to the caf-cline fork as a minimal .gitmodules-only change (§11.4.30-clean).

Error handling: depth-bounded; cycle guard via visited-set on normalized url; any own-org nested chain is a **hard report item** (release-blocker per CONST-051), never auto-rewritten without operator sign-off.

Anti-bluff Challenge (caf_resolve_nested.challenge.sh): build a temp parent with a nested .gitmodules containing one 3rd-party + one fake own-org url; run the script --dry-run; assert the 3rd-party row is classified FORK: and the own-org row PULL_TO_ROOT/CONST051; flip the fixture (move the own-org dep to root) and assert no CONST051 emitted. Evidence under docs/qa/<run-id>/.

3. Submodule-swap procedure + config re-export/install/validate tie-in

After forks exist and are seeded (§2.1) and MAP_FILE is populated:

1. **Rewrite .gitmodules urls** (idempotent, per-line, NOT a blanket git add -A per §11.4.30):

```

while IFS='|' read -r name upstream fork_gh fork_gl; do
    git config -f .gitmodules "submodule.cli_agents/$name.url" "$fork_gh"
done < "$MAP_FILE"

```

(Skip claude-code-source; it stays on its GitLab mirror.)

2. git submodule sync --recursive — propagates the new urls into .git/config and each submodule's .git/config.
3. git submodule set-url is an alternative for individual entries; sync is the bulk path.
4. **Nested swap:** for cline, the nested evals/cline-bench url is rewritten **inside the caf-cline fork's .gitmodules** (not the parent meta-repo), then that fork commits the change.
5. **Re-export + install + validate tie-in:** the existing config/governance pipeline is re-run so the new remotes propagate:
 - ./scripts/init-submodules.sh (re-init against swapped urls),
 - ./scripts/propagate-governance.sh (cascade Constitution/CLAUDE/AGENTS into the new forks — each fork gets the §CONST-047/051 governance siblings, a permitted minimal commit),

- `./scripts/verify-governance-cascade.sh` (confirm anchors present in every fork),
- `install_upstreams` from each fork root **if** it has an `upstreams/` recipe dir (CONST-056),
- a `caf_validate.sh` post-step asserting every `cli_agents/*` url in `.gitmodules` now resolves to `vasic-digital/caf-*` (except the flagged skip) AND `git ls-remote` succeeds for each — captured-evidence gate, not a grep-only pass.

6. **Commit discipline:** the `.gitmodules` rewrite + submodule pointer updates land in ONE meta-repo commit; CONTINUATION updated in the same commit (CONST-044). Forks receive only `.gitignore/governance/.gitmodules-nested` minimal commits — never upstream code edits.

4. Risks

1. **API rate limits.** ~50 forks × 2 providers = up to ~100 create/fork calls plus per-fork clone/fetch. GitHub REST fork endpoint is rate-limited and **async** (fork readiness is eventually-consistent — `gh repo view` may 404 briefly after `gh repo fork`). Mitigation: exponential backoff on 403/429, `gh api rate_limit` polling, poll-until-ready before wiring remotes, resumable `--only` retry from partial `MAP_FILE`.
 2. **Scale (~50 repos) + disk/time.** Mirror-clone fallback for non-forkable upstreams (HTTPS-only like `cline-bench`, or archived repos) is bandwidth/disk heavy (`warp`, `superset`, `amazon-q` are large). Mitigation: `--workdir` on a roomy volume, `--only` batching, background execution per §11.4.89, cleanup trap.
 3. **GitHub + GitLab dual-remote divergence.** Two mirrors of each fork can drift if a push lands on one and fails the other. Mitigation: `caf_update_merge.sh` fans out to **all** provider remotes every run and logs `PUSHFAIL` per-remote; a `caf_validate.sh` parity check compares `git ls-remote` tips across providers. Fast-forward-only pushes keep both linear (§11.4.113).
 4. **Recursive depth / hidden nested deps.** Only `cline` nests today (depth 1), but upstream repos can add submodules at any time, and some use HTTPS (Rule 3 violation if copied verbatim). Mitigation: `--recursive --depth 3 scan` in `caf_resolve_nested_submodules.sh`, url normalization to SSH, cycle guard, and a CONST-051 hard-report for any nested own-org chain.
 5. **Keeping forks' OWN upstream code unchanged.** We do not control upstream source and must not diverge it (merges would conflict forever, and we'd lose the "track upstream" property). Mitigation: forks receive ONLY `.gitignore` + governance siblings + nested-`.gitmodules` rewrites — all non-source, §11.4.30-clean; `caf_update_merge.sh` merges upstream **into** the fork (never the reverse), conflicts park for review rather than auto-resolve, and the merge-onto-latest-main discipline (§11.4.113) guarantees pushes stay fast-forward so no upstream history is ever rewritten.
 6. `claude-code-source` **mis-fork risk.** It is already an own private GitLab mirror; a naive "fork everything" would create a pointless `caf-claude-code-source`. Mitigation: `caf_is_own_org` + default `--exclude claude-code-source` skip-guard, asserted by the §2.1 Challenge's negative case.
 7. **Naming collisions / CONST-052 case.** Fork names preserve the existing kebab submodule dir names (`caf-gpt-engineer`) for upstream traceability; this is intentional and consistent with the kebab paths already in `.gitmodules`. No two `cli_agents/*` names collide under the `caf-` prefix (verified against the 50-entry list).
-

Executive summary (12 lines)

1. `.gitmodules` declares **50** `cli_agents/*` third-party submodule entries (parsed verbatim from the root manifest).
2. **Fork mapping count = 50 forks**: 49 top-level `cli_agents/*` → `vasic-digital/caf-<name>`, plus **1 nested** 3rd-party (`cline-bench`).
3. **1 entry skipped**: `cli_agents/claude-code-source` is already an own/private GitLab mirror (`gitlab.com:ilos85vasic/ccode-private`) — not forked.
4. **1 nested dep found** by full sweep: `cli_agents/cline/.gitmodules` → `evals/cline-bench` (`https://github.com/cline/cline-bench.git`, HTTPS → must become SSH in the fork).
5. Representative agents (`claude-code`, `qwen-code`, `aider`, `plandex`, `crush`, `gemini-cli`) have **no** nested submodules; max fleet nesting depth = **1**; **no** nested own-org chains under `cli_agents/*` (CONST-051 clean).
6. One org-prefixed SSH alias noted (`org-14957082@github.com` on OpenAI-Cookbook) is in `cli_agents_resources/` — out of scope; parser must normalize `org-NNNN@github.com:` → `github.com:`.
7. **Three reusable scripts** (in `constitution/scripts/`, sharing `caf_lib.sh`, fully parameterized — `org/prefix/src-dir/providers/branch` all flags, nothing hardcoded):
8. `caf_fork_all.sh` — fork-or-create on GitHub+GitLab, wire `origin(fork)` + `upstream(original)`, record `MAP_FILE`; idempotent, rate-limit backoff, mirror-push fallback for non-forkable/HTTPS upstreams (empty-target seed, §11.4.113-safe).
9. `caf_update_merge.sh` — fetch + merge upstream `main/master` into each fork, fast-forward-preferred, **never force**, conflicts parked for review, fan-out push to all providers.
10. `caf_resolve_nested_submodules.sh` — recursive (`--depth 3`) nested-dep classifier: 3rd-party → fork + SSH-rewrite; own-org → hard CONST-051 pull-to-root report (never auto-rewritten).
11. Each script ships an **anti-bluff Challenge** (throwaway dir, tiny fixture repo, assert real remotes / fast-forward / `no--force` in git trace, evidence under `docs/qa/<run-id>/`); `swap = .gitmodules per-line url rewrite + git submodule sync --recursive + governance re-cascade + caf_validate.sh resolve check`.
12. Top risks: GitHub/GitLab rate limits + async fork readiness, ~50-repo scale/disk, dual-remote drift, recursive/HTTPS nested deps, and keeping forks' upstream **code** untouched (only `.gitignore/governance/nested-.gitmodules` commits; merge-into-fork only, §11.4.30 + §11.4.113).